

SQL Aggregation Functions, GROUP BY, HAVING, WHERE vs HAVING, and NULL Handling

Let's understand these concepts using a single table.

Step 1: Create the Table

None

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    department VARCHAR(30),  
    salary DECIMAL(10,2),  
    age INT,  
    bonus DECIMAL(10,2)  
);
```

Step 2: Insert Sample Data

None

```
INSERT INTO employees  
VALUES  
(101, 'Alice', 'HR', 40000, 25, 5000),  
(102, 'Bob', 'Sales', 50000, 30, NULL),  
(103, 'Charlie', 'Sales', 55000, 29, 7000),  
(104, 'David', 'IT', 60000, 35, NULL),  
(105, 'Eva', 'IT', 65000, 31, 8000),  
(106, 'Frank', 'HR', 45000, 27, NULL),  
(107, 'Grace', 'Sales', 70000, 32, 10000);
```

1. COUNT()

COUNT() returns the number of rows.

COUNT(*)

Counts all rows.

None

```
SELECT COUNT(*)  
FROM employees;
```

Output

COUNT(*)

7

COUNT(column)

Counts only NON-NULL values.

None

```
SELECT COUNT(bonus)  
FROM employees;
```

Output

COUNT(bonus)

4

Because three bonus values are NULL.

COUNT(DISTINCT)

None

```
SELECT COUNT(DISTINCT department)
FROM employees;
```

Output

COUNT

3

Departments are

- HR
- Sales
- IT

2. SUM()

Returns the total.

None

```
SELECT SUM(salary)
FROM employees;
```

Output

None

385000

Department-wise salary

None

```
SELECT department,
       SUM(salary)
```

```
FROM employees  
GROUP BY department;
```

Output

Department	Total Salary
HR	85000
Sales	175000
IT	125000

3. AVG()

Returns average.

```
None  
SELECT AVG(salary)  
FROM employees;
```

Output

```
None  
55000
```

Department-wise average

```
None  
SELECT department,  
       AVG(salary)  
FROM employees  
GROUP BY department;
```

Output

Department	Average Salary
HR	42500
Sales	58333.33
IT	62500

4. MIN()

Lowest value.

```
None
SELECT MIN(salary)
FROM employees;
```

Output

```
None
40000
```

Department-wise minimum

```
None
SELECT department,
       MIN(salary)
FROM employees
GROUP BY department;
```

Output

Department	Minimum Salary
------------	----------------

HR	40000
Sales	50000
IT	60000

5. MAX()

Largest value.

None

```
SELECT MAX(salary)
FROM employees;
```

Output

None

```
70000
```

Department-wise maximum

None

```
SELECT department,
       MAX(salary)
FROM employees
GROUP BY department;
```

Output

Department	Maximum Salary
HR	45000
Sales	70000
IT	65000

Multiple Aggregate Functions Together

None

```
SELECT
COUNT(*) AS Employees,
SUM(salary) AS Total_Salary,
AVG(salary) AS Average_Salary,
MIN(salary) AS Lowest_Salary,
MAX(salary) AS Highest_Salary
FROM employees;
```

Output

Employee s	Total	Averag e	Min	Max
7	385000	55000	40000	70000

GROUP BY

What is GROUP BY?

GROUP BY combines rows having the same values.

Suppose we want department-wise statistics.

None

```
SELECT department,
COUNT(*) AS TotalEmployees
FROM employees
GROUP BY department;
```

Output

Department	Employees
HR	2
Sales	3
IT	2

Another Example

None

```
SELECT department,  
AVG(salary)  
FROM employees  
GROUP BY department;
```

Output

Each department becomes one group.

None

```
HR  
Sales  
IT
```

Average is calculated separately.

GROUP BY Execution Order

None

```
FROM
```

↓

```
WHERE
```

↓

GROUP BY



Aggregate Functions



HAVING



SELECT



ORDER BY

HAVING

HAVING filters groups.

WHERE filters rows.

Suppose we need departments whose average salary is above ₹50,000.

None

```
SELECT department,  
AVG(salary) AS AvgSalary  
FROM employees  
GROUP BY department  
HAVING AVG(salary) > 50000;
```

Output

Department	Average Salary
------------	----------------

Sales	58333
-------	-------

IT	62500
----	-------

HR is removed.

Another Example

Departments having more than two employees.

None

```
SELECT department,  
COUNT(*)  
FROM employees  
GROUP BY department  
HAVING COUNT(*) > 2;
```

Output

Department	Employee s
------------	---------------

Sales	3
-------	---

WHERE vs HAVING

Suppose we need

Departments where employees earn above 45000.

WHERE

None

```
SELECT department,  
AVG(salary)  
FROM employees  
WHERE salary > 45000  
GROUP BY department;
```

First removes rows.

Remaining employees

None

```
Bob  
Charlie  
David  
Eva  
Grace
```

Then average is calculated.

HAVING

None

```
SELECT department,  
AVG(salary)  
FROM employees  
GROUP BY department  
HAVING AVG(salary)>50000;
```

First averages are calculated.

Then departments are filtered.

Comparison Table

WHERE

Filters rows

Executed before GROUP BY

Cannot use aggregate functions

Faster

HAVING

Filters groups

Executed after GROUP BY

Can use aggregate functions

Slightly slower

Example

WHERE

None

```
SELECT *  
FROM employees  
WHERE salary>50000;
```

Returns employees.

HAVING

None

```
SELECT department,  
COUNT(*)  
FROM employees  
GROUP BY department  
HAVING COUNT(*)>=2;
```

Returns departments.

NULL Handling in Aggregation

Our bonus column contains NULL.

Employee	Bonus
Alice	5000
Bob	NULL
Charlie	7000
David	NULL
Eva	8000
Frank	NULL
Grace	10000

COUNT(*)

Counts every row.

None

```
SELECT COUNT(*)  
FROM employees;
```

Output

None

7

COUNT(column)

Ignores NULL.

None

```
SELECT COUNT(bonus)
FROM employees;
```

Output

None

4

SUM()

NULL values are ignored.

None

```
SELECT SUM(bonus)
FROM employees;
```

Output

None

30000

Calculation

None

5000

7000

8000

10000

30000

AVG()

Ignores NULL.

None

```
SELECT AVG(bonus)
FROM employees;
```

Output

None

7500

Average

None

30000/4

NOT

None

30000/7

MIN()

None

```
SELECT MIN(bonus)
FROM employees;
```

Output

None

5000

MAX()

None

```
SELECT MAX(bonus)
FROM employees;
```

Output

None

```
10000
```

Replacing NULL with 0

Using `COALESCE()` (works in MySQL, PostgreSQL, SQL Server)

None

```
SELECT emp_name,
COALESCE(bonus,0) AS Bonus
FROM employees;
```

Output

Employee	Bonus
Alice	5000
Bob	0
Charlie	7000
David	0
Eva	8000
Frank	0
Grace	10000